The Compression Floor: How Few-Step Decoding Collapses in Small Block-Diffusion Language Models

Anonymous Author(s)
Affiliation
Address
email

Abstract

Block-diffusion language models accelerate generation by predicting multiple tokens per denoising step, but the limits of this compression remains poorly understood. Therefore, we study SDAR-4B-Chat with block size **32** on a mathematical benchmark and identify a sharp **compression floor** with three properties. First, quality of sample generation exhibits a phase transition rather than gradual degradation: across pooled schedules, **60%** of outputs remain less repeated, while **23%** collapse into severe repetition loops, with only **9%** in intermediate regime. Second, supervised fine-tuning (SFT) can worsen compression tolerance. Dense supervision on revealed positions reduces one-token-per-step accuracy from **48%** to **8%**, whereas a matched masked-only objective held it at **20%**. SFT also reduces the usable compression range from four to two tokens per step. Third, on-policy self-distillation inherits this floor through trajectory yield: at temperature **0.9**, the student produced **0** usable trajectories in **28** attempts, while temperature **0.3** with a larger generation budget yielded **38/50**. We replicate this yield collapse on a second model family, suggesting the phenomenon is not architecture-specific. Together, these results show that few-step diffusion language modeling is governed not only by the number of tokens revealed per step, but also by how supervision and sampling interact with the model's compression floor. Our findings highlight supervision density and sampling strategy as two practical levers for making accelerated diffusion-LM training reliable. The code is available at `https://github.com/diffuserresearcher-glitch/Compression_Floor--Diffusion_Language_Model`.

1 Introduction

Diffusion language models generate text by iteratively revealing masked tokens, which allows several tokens to be committed in one forward pass. Block-diffusion variants such as SDAR [Cheng et al., 2025] and TraDo [Wang et al., 2025] decode a fixed block in parallel while remaining autoregressive across blocks. The appeal is speed: fewer denoising steps per block means fewer forward passes.

This paper measures that cost directly on SDAR-4B-Chat with block size 32, on mathematical reasoning problems from DeepMath-103K [He et al., 2025]. We hold one 40-problem benchmark fixed and vary the number of tokens revealed per step from 1 to 32. We then ask two follow-up questions that matter for anyone trying to train such a model to decode faster. Does supervised fine-tuning (SFT) move the floor, and in which direction? And can on-policy self-distillation, which trains a fast student against a better-informed teacher built from the same weights [Zhao et al., 2026], recover the lost accuracy?

Submitted to 40th Conference on Neural Information Processing Systems (NeurIPS 2026). Do not distribute.

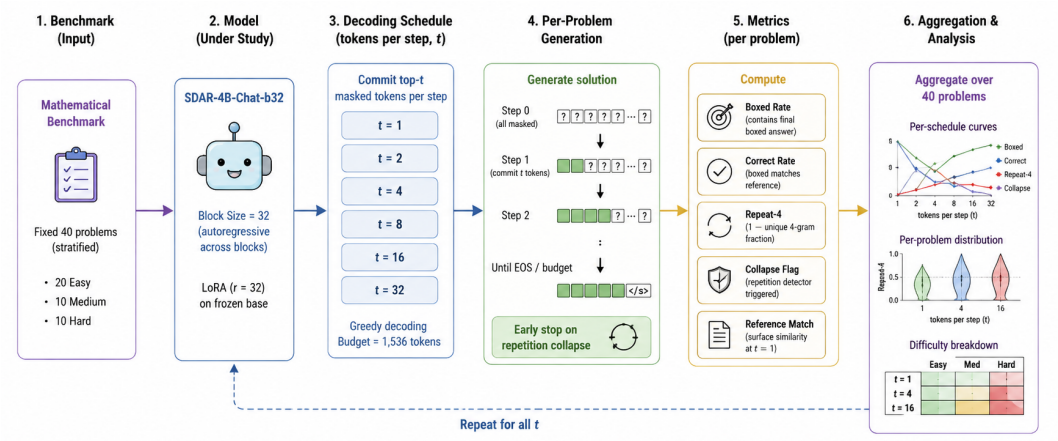

Figure 1: **The Compression Floor Evaluation Pipeline.** We evaluate SDAR-4B-Chat-b32 with block size 32 on a fixed mathematical benchmark across decoding schedules with $t \in \{1, 2, 4, 8, 16, 32\}$ tokens committed per denoising step. For each schedule, the model generates solutions using greedy decoding until EOS or a 1,536-token budget, with early stopping upon repetition collapse. We compute boxed rate, correct rate, repeat-4, collapse rate, and reference match for each problem, then aggregate results across schedules and difficulty levels. The same evaluation protocol is applied to the base model, SFT variants, and on-policy self-distillation experiments.

The failure mode at aggressive schedules is not a smooth quality decline that training can polish away. It is a bimodal collapse into repetition loops. SFT does not widen the usable schedule range. Every variant we trained narrowed it, and the damage depends strongly on supervision density.

Self-distillation never reaches the training stage unless the student can produce usable trajectories at all, and whether it can is decided by sampling parameters, not by the method.

Our contributions are the following measurements, all on one fixed benchmark:

- We show that few-step failure in a small block-diffusion model is a per-sample phase transition. Individual outputs are either fluent or collapsed, and mean metrics degrade only because the collapsed fraction grows (Section 3.1).
- We give a controlled comparison of SFT supervision density. Supervising all block positions destroys the model monotonically across a three-stage schedule curriculum. Supervising only masked positions, on matched schedules and data scale, roughly halves the damage and changes its structure, but does not restore base accuracy (Section 3.2).
- We identify trajectory yield as the operating precondition of on-policy self-distillation for diffusion models, show that it is controlled by sampling temperature and budget, and replicate the yield collapse on a second model family (Section 3.3).

Recent work has pursued few-step diffusion language modeling along two complementary directions. Both masked and block-diffusion hybrids such as SDAR and TraDo reduce sequential depth by revealing multiple tokens in parallel, while training-free methods such as Fast-dLLM further accelerate inference without modifying the model. These approaches expose a fundamental trade-off between generation speed and the number of denoising steps, and recent analyses attribute few-step degradation to errors introduced when multiple tokens must be predicted simultaneously. A second line of work attempts to recover this lost quality through distillation. T3D distills on rollout trajectories produced under the target decoding procedure, while OPSD introduces on-policy self-distillation, in which teacher and student share weights and the teacher conditions on privileged information while both are evaluated on the student's own rollout. However, these approaches implicitly require the student to produce sufficiently useful trajectories for training. In contrast, we focus on the regime where this prerequisite itself breaks down: we characterize the compression floor at the level of individual generations and study how supervised fine-tuning and sampling choices move or expose this floor. Thus, rather than proposing another acceleration or distillation method, our work provides

an empirical characterization of when few-step diffusion generation remains trainable and when it collapses.

2 Experimental Setup

Model and data: We conduct our experiments on SDAR-4B-Chat-b32 [Cheng et al., 2025], a 4B-parameter block-diffusion language model with block size 32 (b32) and block size 16 (b16), obtained by adapting an autoregressive base model. Training and evaluation instances are drawn from DeepMath-103K [He et al., 2025], which provides verified final answers together with three R1-generated solution traces per problem. We use a fixed held-out benchmark of 40 problems, stratified by dataset difficulty into 20 easy, 10 medium, and 10 hard instances. This subset was selected to provide balanced coverage across difficulty levels while keeping the extensive multi-schedule decoding and training experiments computationally feasible. Unless otherwise specified, fine-tuning is performed with LoRA adapters of rank 32 while keeping the base model frozen.

Decoding protocol: For a block containing 32 masked positions, we decode t tokens per denoising step by committing the t highest-confidence masked positions. Thus, $t = 1$ requires 32 denoising steps per block, whereas $t = 32$ completes the block in a single step. We evaluate schedules spanning $t \in \{1, 2, 4, 8, 16\}$ and use greedy decoding unless otherwise stated. Generations are limited to 1,536 tokens.

Evaluation metrics: We measure both task completion and generation stability. *Boxed rate* is the fraction of generations containing a final boxed answer, while *correct rate* additionally requires the extracted boxed answer to match the verified reference answer. To quantify degenerative repetition, we use *Repeat-4*, defined as one minus the fraction of unique 4-grams in a generation. Thus, lower values indicate greater diversity, while values approaching one correspond to severe repetition.

Evaluation protocols: We use two evaluation harnesses for distinct experimental comparisons while keeping the underlying benchmark and prompts fixed. The decoding-grid experiments in Section 3.1 and the SFT control comparison in Section 3.2 use a common grid-based evaluator with an explicit repetition-collapse stopping criterion. The three-stage curriculum comparison in Section 3.2 uses the training harness associated with the corresponding fine-tuning runs and reports a more permissive string-match accuracy. Because the stopping and matching criteria differ, results from the two protocols are treated as separate measurements and are never combined within a single comparison. Where needed, the base model is evaluated independently under each protocol to provide the appropriate reference point.

3 Results

3.1 The floor is a per-sample phase transition

Table 1 reports the base model at block 32 across schedules. Accuracy holds up to $t = 2$, degrades at $t = 4$, and reaches zero between $t = 4$ and $t = 8$. The collapse-detector stop fraction mirrors this cliff, rising from 0 at $t \leq 2$ to 0.62 at $t = 16$. Block size 16 shows the identical cliff location (correct 0.20 at $t = 4$, 0.00 at $t = 8$; full grid omitted for space).

Table 1: Base SDAR-4B-Chat at block sizes 16 and 32 across decoding schedules (40 problems per cell, greedy). The accuracy cliff sits between 4 and 8 tokens per step. Collapse is the fraction of generations halted by the repetition detector.

Block	Metric	t=1	t=2	t=4	t=8	t=16
16	correct	.425	.375	.200	.000	.000
	repeat-4	.152	.181	.403	.579	.684
	collapse	.000	.000	.100	.500	.500
32	correct	.375	.250	.150	.000	.000
	repeat-4	.161	.156	.268	.386	.590
	collapse	.000	.000	.050	.275	.625

The mean curves hide the structure. Figure 2 pools per-problem repeat-4 over all schedules. The distribution is bimodal. At block 32, 60% of outputs sit below 0.2, 23% sit above 0.7, and only 9% occupy the middle band from 0.3 to 0.6. Individual generations do not degrade gradually as the schedule coarsens. They either hold together or fall into a loop, and coarser schedules only shift mass between the two modes. This matches the factorization-error account of few-step decoding [Zhang et al., 2026]: once simultaneously committed tokens must be predicted independently, an unlucky commitment locks the block, and everything after it, into repetition. The cost is also difficulty-selective. At block 32 and $t = 1$, medium-tier boxed rate is 0.60, and it is the first tier to reach zero as t grows.

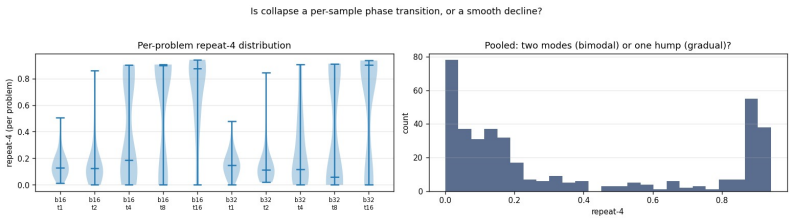

Figure 2: Per-problem repeat-4 for the base model, pooled over schedules. The distribution has a fluent mode and a collapsed mode with little mass between them. Cell means rise with tokens per step only because the collapsed fraction grows.

3.2 SFT narrows the window, and supervision density and dataset size decide how much

We fine-tuned the model three ways on DeepMath solutions, always with LoRA, and benchmarked every checkpoint on the fixed 40 problems.

SFT shifts the cliff inward. A three-stage SFT run that supervised all 32 block positions per denoising state was re-benchmarked across the full schedule grid (Figure 3). Relative to the base model at block 32, its ceiling fell (correct 0.375 to 0.175 at $t = 1$) and its cliff moved one octave earlier, from between $t = 4$ and $t = 8$ to between $t = 2$ and $t = 4$. Redundancy rose at every schedule. Reference-solution coverage at $t = 1$ rose above base (0.506 versus 0.446) while accuracy fell, so the model learned to imitate reference phrasing at the cost of finishing problems. Medium-tier accuracy was zero at every schedule, including $t = 1$.

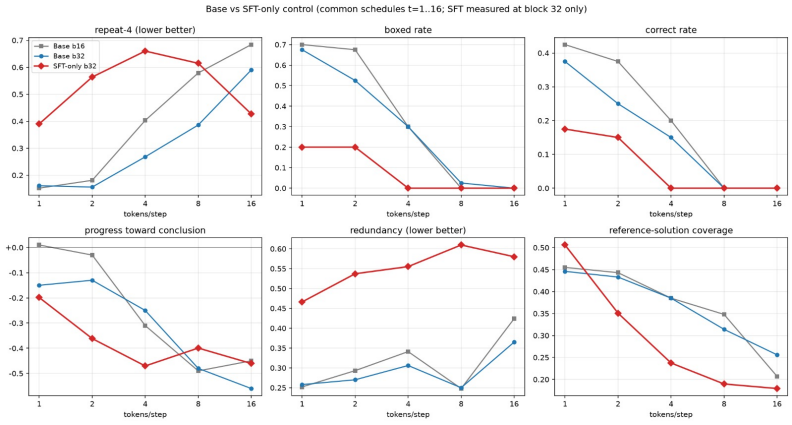

Figure 3: Base model versus the dense-supervision SFT control at block 32, across common schedules. SFT lowers the accuracy ceiling, moves the collapse cliff one octave earlier, and raises redundancy everywhere, while increasing surface similarity to reference solutions at $t = 1$.

A three-stage curriculum lets us ask whether this collapse is the objective, or the data. We ran two matched three-stage curricula (stages at 4, 2, and 1 tokens per step, 500 problems and 2 epochs per stage) that differ in both the loss and the data schedule. The dense arm supervises every block position at each denoising state and reuses the *same* 500-problem pool across all three stages,

giving each problem 6 effective epochs of exposure. The masked-only arm supervises only the still-masked positions, the standard masked-diffusion objective [Sahoo et al., 2024], and draws a *fresh* 500-problem pool for each stage (1,500 problems total, no reuse), specifically to remove the overfitting we observed in the dense arm. Because the two arms differ in loss and data simultaneously, this is not a clean ablation of the objective; it directly tests whether fixing the data-reuse problem is enough to recover base performance.

It is not. Table 2 shows that dense supervision degrades monotonically and severely as the curriculum progresses (48% to 8% correct at $t = 1$), consistent with the repeated pool driving memorization-then-collapse rather than a property of dense supervision per se. Removing the reuse in the masked-only arm recovers 10 to 18 points at every stage and roughly halves the repetition increase, confirming that data reuse explains a large part of the dense arm's failure. But the masked-only arm's best checkpoint still trails the base model by 18 points at $t = 1$ (30% versus 48%), despite using fresh, non-repeated data at every stage. So while overfitting on a reused pool is the dominant driver of the dense arm's collapse, it does not account for all of the degradation: fine-tuning on this curriculum degrades the model even when overfitting is controlled for, and we cannot attribute that residual gap to the loss function alone given the remaining confounds below.

One might object that the dense arm's disadvantage is simply an artifact of our attribution, not a property of the objective: perhaps dense supervision would match or exceed masked-only if it too were given 1,500 unique problems instead of 500 reused ones. Stage 1 speaks to this. At stage 1, neither arm has yet reused any data: the dense arm has seen its 500-problem pool for the first time, and the masked-only arm has seen its first fresh 500-problem pool, so the two checkpoints are trained on equally novel, equally sized data at this point in the curriculum. Dense is nonetheless already well behind masked-only (20% versus 30% correct at $t = 1$), and the gap widens at every subsequent stage as the dense arm's pool reuse compounds. That the disadvantage is present before any reuse has occurred, and grows monotonically once it does, is consistent with dense supervision being a poor objective in its own right rather than merely a data-starved one; more data would need to reverse an already-negative and worsening trend, not just extend a currently-even one.

Two further observations hold regardless of this attribution question. First, the medium tier collapses to zero under nearly every fine-tuned checkpoint in both arms, so difficulty-selective fragility appears in both the overfit and non-overfit conditions. Second, the masked-only arm shows a schedule crossover absent under dense supervision: its final stage is the best checkpoint at $t = 2$ (30%) and the worst at $t = 1$ (20%), so the residual failure changes shape, not just size.

Table 2: Supervision-density comparison on the fixed benchmark (training-notebook evaluator; the base row uses the same evaluator). Accuracy is the lenient match rate; repeat-4 is reported at t=1. The masked-only objective recovers 10 to 18 points at every stage but does not restore base accuracy.

	Dense (all positions)		Masked-only		
Checkpoint	t=1	repeat-4	t=1	repeat-4	Δ acc.
base	48%	.185	48%	.185	–
stage 1 (t=4)	20%	.436	30%	.274	+10
stage 2 (t=2)	10%	.608	28%	.247	+18
stage 3 (t=1)	8%	.497	20%	.350	+12

3.3 Self-distillation inherits the floor through trajectory yield

On-policy self-distillation requires the student model to generate useful trajectories before training can begin. For our math tasks, we consider a rollout useful if it reaches a boxed final answer within the generation budget. Table 3 shows that this yield depends strongly on the sampling settings. With temperature 0.9 and a 768-token budget, we obtained 0 usable rollouts from 28 attempts with SDAR-4B-Chat. When we lowered the temperature to 0.3 and increased the budget to 1,536 tokens, we obtained 38 usable rollouts from 50 attempts, including 13 correct ones. The failed rollouts were not necessarily poor generations: their mean repeat-4 score was only 0.17, and none produced an incorrect boxed answer. They simply failed to reach a conclusion within the allowed budget. We observed a similar issue with another model, TraDo-4B [Wang et al., 2025], where temperature 0.9 produced no boxed rollouts in the first 36 attempts. These results show that trajectory yield is strongly affected by the sampling settings and should be measured before running self-distillation.

This issue is also important for existing diffusion OPSD methods. [Luo et al., 2026] build the self-teacher from the student's own on-policy trajectories, which are the same type of trajectories that we screen here. On GSM8K-scale tasks, they report that using a pass@k budget of k=8 works better than k=1, with k=1 showing a measurable performance drop (Table 8 of their work). Our results provide a possible explanation for why a larger sampling budget is useful: usable trajectories can be rare under some sampling settings. In our SDAR-4B-Chat-b32 experiment, temperature 0.9 produced zero usable trajectories with a 768-token budget. In such a case, simply increasing the number of samples does not solve the problem unless the generation settings also allow the model to reach a conclusion. The required sampling budget therefore depends on the model and the sampling regime. We argue that trajectory yield should be measured and reported before applying self-distillation, because a distillation method that requires usable student trajectories cannot work when those trajectories are not produced in the first place.

Table 3: Trajectory yield of screening rollouts under different sampling parameters. Usable means the rollout contains a boxed final answer. Yield, the precondition for on-policy self-distillation, is set by sampling parameters rather than by model capability.

Model	Temp. / budget	Usable	Correct	Mean repeat-4
SDAR-4B-Chat-b32	0.9 / 768	0 / 28	0 / 28	.17
SDAR-4B-Chat-b32	0.3 / 1,536	38 / 50	13 / 50	–
TraDo-4B	0.9 / 768	0 / 36	0 / 36	–

4 Limitations and conclusion

Our results have several limitations. We use a single 40-problem benchmark and one random seed, so the accuracy estimates have a binomial uncertainty of about ± 15 points. The results should therefore be viewed as evidence of large effects rather than as precise estimates. All fine-tuning experiments use LoRA on a single 4B model, so the behavior may differ with full fine-tuning or larger models. The supervision-density comparison in Section 3.2 also has differences in data reuse and target cleanliness, so the observed gap cannot be attributed to the training objective alone. We use the same evaluator consistently within each comparison and never mix the two evaluators. Finally, we do not yet measure the performance of the model after self-distillation, so our results establish the conditions needed for distillation but not its final benefit.

There are also two important differences between the dense and masked-only training arms. The masked-only arm removed raw reasoning traces and kept end-of-sequence tokens intact, making its training targets cleaner than those of the dense arm. In addition, most reference solutions are longer than the training length limit. As a result, both arms contain truncated targets without a stop token. This may contribute to the non-termination behavior observed across all fine-tuned checkpoints, including those that are not strongly overfit. Because of these confounding factors, Table 2 should be interpreted carefully. The main result is that fine-tuning hurts the base model under both training objectives. The particularly strong degradation in the dense arm is likely partly caused by reusing the same small data pool across stages. However, even after removing this data-reuse issue, the masked-only arm still performs substantially worse than the base model. We therefore cannot determine how much of this remaining gap is caused by the training objective or by differences in target cleanliness.

Overall, we find that the compression limit of a small block-diffusion language model is not a smooth decline in performance. Instead, individual samples undergo a sharp transition from successful generation to repetitive collapse. Fine-tuning makes this limit worse across all variants we tested. The density of supervision is an important factor: masked-only supervision reduces the damage compared with dense supervision, but does not remove it. We also find that self-distillation requires a sufficient number of usable trajectories, and that this trajectory yield can be strongly affected by the sampling temperature and generation budget. These results suggest that future work on few-step diffusion language model distillation should report per-sample collapse rates, supervision density, and trajectory yield before evaluating the final distillation performance.

References

Marianne Arriola, Aaron Gokaslan, Justin T. Chiu, Zhihan Yang, Zhixuan Qi, Jiaqi Han, Subham Sekhar Sahoo, and Volodymyr Kuleshov. Block diffusion: Interpolating between autoregressive and diffusion language models. *arXiv preprint arXiv:2503.09573*, 2025.

Shuang Cheng, Yihan Bian, Dawei Liu, Linfeng Zhang, Qian Yao, Zhongbo Tian, Wenhai Wang, Qipeng Guo, Kai Chen, Biqing Qi, and Bowen Zhou. SDAR: A synergistic diffusion-autoregression paradigm for scalable sequence generation. *arXiv preprint arXiv:2510.06303*, 2025.

Justin Deschenaux and Çağlar Gülçehre. Beyond autoregression: Fast LLMs via self-distillation through time. In *International Conference on Learning Representations (ICLR)*, 2025. arXiv:2410.21035.

Zhiwei He, Tian Liang, Jiahao Xu, Qiuzhi Liu, Xingyu Chen, Yue Wang, Linfeng Song, Dian Yu, Zhenwen Liang, Wenxuan Wang, Zhuosheng Zhang, Rui Wang, Zhaopeng Tu, Haitao Mi, and Dong Yu. DeepMath-103K: A large-scale, challenging, decontaminated, and verifiable mathematical dataset for advancing reasoning. *arXiv preprint arXiv:2504.11456*, 2025.

Yifu Luo, Zeyu Chen, Haoyu Wang, Xinhao Hu, Yuxuan Zhang, Zhizhou Sha, and Shiwei Liu. Learning from the self-future: On-policy self-distillation for dllms. *arXiv preprint arXiv:2606.18195*, 2026.

Shen Nie, Fengqi Zhu, Zebin You, Xiaolu Zhang, Jingyang Ou, Jun Hu, Jun Zhou, Yankai Lin, Ji-Rong Wen, and Chongxuan Li. Large language diffusion models. *arXiv preprint arXiv:2502.09992*, 2025.

Subham Sekhar Sahoo, Marianne Arriola, Yair Schiff, Aaron Gokaslan, Edgar Marroquin, Justin T. Chiu, Alexander Rush, and Volodymyr Kuleshov. Simple and effective masked diffusion language models. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024. arXiv:2406.07524.

Yinjie Wang, Ling Yang, Bowen Li, Ye Tian, Ke Shen, and Mengdi Wang. Revolutionizing reinforcement learning framework for diffusion large language models. *arXiv preprint arXiv:2509.06949*, 2025.

Chengyue Wu, Hao Zhang, Shuchen Xue, Zhijian Liu, Shizhe Diao, Ligeng Zhu, Ping Luo, Song Han, and Enze Xie. Fast-dLLM: Training-free acceleration of diffusion LLM by enabling KV cache and parallel decoding. *arXiv preprint arXiv:2505.22618*, 2025.

Tunyu Zhang, Xinxi Zhang, Ligong Han, Haizhou Shi, Xiaoxiao He, Zhuowei Li, Hao Wang, Kai Xu, Akash Srivastava, Hao Wang, Vladimir Pavlovic, and Dimitris N. Metaxas. T3D: Few-step diffusion language models via trajectory self-distillation with direct discriminative optimization. *arXiv preprint arXiv:2602.12262*, 2026.

Siyan Zhao, Zhihui Xie, Zichen Liu, Kaiwen Huang, Tianyu Pang, Changyu Chen, and Aditya Grover. Self-distilled reasoner: On-policy self-distillation for large language models. *arXiv preprint arXiv:2601.18734*, 2026.

A Additional Related Work

Block-diffusion language models and few-step decoding. Masked diffusion language models [Sahoo et al., 2024, Nie et al., 2025] and block-diffusion hybrids [Arriola et al., 2025, Cheng et al., 2025, Wang et al., 2025] trade sequential depth for parallel width. Training-free accelerators such as Fast-dLLM [Wu et al., 2025] push the same trade-off at inference time. T3D [Zhang et al., 2026] attributes few-step degradation to factorization error in the token-independent reverse process, which grows as steps shrink. Our work differs in scope: we do not propose an accelerator. We characterize the shape of the failure at small scale, per sample rather than in the mean, and quantify how post-training moves it.

Self-distillation for fewer steps. SDTT [Deschenaux and Gülçehre, 2025] distills a many-step teacher into a few-step student. T3D [Zhang et al., 2026] distills on rollout trajectories produced under the target decoding procedure. OPSD [Zhao et al., 2026] makes distillation on-policy for autoregressive models: teacher and student share weights, the teacher conditions on privileged information, and both are evaluated on the student's own rollout. A concurrent work adapts on-policy self-distillation to diffusion models on LLaDA-8B [Luo et al., 2026]. We differ from all of these in what we measure. Prior work reports post-distillation accuracy on students that produce usable trajectories. We measure the precondition itself: whether a small student near the floor produces trainable trajectories at all, and what controls that.

B Formal Metric Definitions

Boxed rate and correct rate. For a generation g and reference answer $a^\star$, let $\mathrm{box}(g)$ denote the extracted contents of the final `\boxed{}` expression in g, or $\perp$ if none is present. Boxed rate over a set of N problems is $\mathrm{Boxed} = \frac{1}{N}\sum_{i=1}^{N} \mathbb{1}[\mathrm{box}(g_i) \neq \perp]$. Correct rate additionally requires an exact match to the verified reference, $\mathrm{Correct} = \frac{1}{N}\sum_{i=1}^{N} \mathbb{1}[\mathrm{box}(g_i) = a_i^\star]$. Correct rate is a subset of boxed rate by construction.

Repeat-4. For a generation g tokenized into n-grams of length 4, let $U_4(g)$ be the number of distinct 4-grams and $C_4(g)$ the total count of 4-grams. Repeat-4 is $\mathcal{R}_4(g) = 1 - U_4(g)/C_4(g)$. Lower values indicate greater lexical diversity; values approaching 1 indicate that the generation is dominated by a small set of repeated 4-grams.

Collapse flag. A generation is flagged as *collapsed* if the online repetition detector halts decoding before the token budget or EOS is reached. The collapse rate over N problems is the fraction so flagged, $\mathrm{Collapse} = \frac{1}{N}\sum_{i=1}^{N} \mathbb{1}[haltedbyrepetitiondetector]$. Collapse and boxed-completion are mutually exclusive stopping causes; every generation ends in exactly one of the two.

Reference-solution coverage. Let $S(g)$ and $S(a^\star_{trace})$ be the sets of unique content n-grams (stopwords removed) in the generation and in the reference reasoning trace, respectively. Coverage is the fraction of reference n-grams reproduced in the generation, $\mathrm{Coverage}(g) = |S(g) \cap S(a^\star_{trace})| \,/\, |S(a^\star_{trace})|$. Coverage is reported as the mean over the benchmark. Higher coverage indicates closer surface alignment with the reference solution and is measured independently of correctness.

Redundancy. Redundancy quantifies within-generation repetition at the sentence or reasoning-step level, distinct from the token-level Repeat-4 metric. Let $\{s_1, \ldots, s_k\}$ be the segmented reasoning steps of a generation. Redundancy is the mean pairwise similarity across steps, $\mathrm{Redundancy}(g) = k2^{-1}\sum_{j<l} \mathrm{sim}(s_j, s_l)$, where $\mathrm{sim}(\cdot,\cdot)$ is a lexical overlap similarity (unique-token Jaccard) between step pairs. Higher values indicate the generation is looping over near-duplicate reasoning steps rather than advancing.

Progress toward conclusion. This metric scores whether successive reasoning steps move monotonically toward a final boxed answer, as opposed to stalling or reversing. For a generation segmented into steps $s_1, \ldots, s_k$, let $d(s_j)$ be a step-wise progress signal (e.g., proximity of the step's stated intermediate quantities to the final boxed value, or increasing coverage of the reference solution up to step j). Progress is the mean step-to-step change, $\mathrm{Progress}(g) = \frac{1}{k-1}\sum_{j=1}^{k-1}\big[d(s_{j+1}) - d(s_j)\big]$. Positive values indicate the generation is converging toward a conclusion; negative values indicate stalling, backtracking, or drift, which is common in collapsed generations that never reach a boxed answer.

Trajectory yield. For on-policy self-distillation screening (Section 3.3), a rollout is *usable* if $\mathrm{box}(g) \neq \perp$ within the sampling budget. Yield at a given temperature τ and budget B is $\mathrm{Yield}(\tau, B) = \frac{1}{M}\sum_{i=1}^{M} \mathbb{1}[\mathrm{box}(g_i) \neq \perp]$ over M screening attempts. Yield is a precondition for downstream distillation and is measured independently of correctness.

C Hyperparameters and Training Details

C.1 SFT Training

We use the same base model and LoRA configuration for both the dense (supervise-all) and masked-only SFT arms. Unless otherwise stated, all hyperparameters are kept identical between the two arms. The main difference is the supervision objective: the dense arm supervises all block positions, while the masked-only arm supervises only the positions that remain masked.

Table 4: Hyperparameters used for the dense and masked-only SFT arms. All listed optimization and LoRA settings are identical across the two arms.

Hyperparameter	Dense (supervise-all)	Masked-only
Base model	SDAR-4B-Chat-b32	SDAR-4B-Chat-b32
Block size	32	32
LoRA rank	32	32
LoRA α	64	64
LoRA dropout	0.05	0.05
LoRA target modules	q/k/v/o_proj, gate/up/down_proj	q/k/v/o_proj, gate/up/down_proj
Optimizer	AdamW	AdamW
Learning rate	1×10^{-5}	1×10^{-5}
LR schedule	cosine + warmup	cosine + warmup
Warmup steps	10	10
Epochs per stage	2	2
Number of stages	3	3
Batch size	1	1
Gradient accumulation	4	4
Effective batch size	4	4
Maximum target tokens	384	384
GPU	H-100 (12 GB)	Nvidia RTX Pro (102 GB)

The three SFT stages use the same overall training setup, with the curriculum progressing through $t = 4$, $t = 2$, and $t = 1$ tokens per step. Each stage is trained for two epochs. The effective batch size is four sequences, obtained from a per-step batch size of one and gradient accumulation over four steps.

The dense and masked-only arms were implemented with the same optimizer, learning rate, scheduler, LoRA configuration, batch mechanics, and target length. The masked-only arm differs in which positions contribute to the loss. This controlled setup allows us to compare the effect of supervision density, while the differences in data sourcing and target preprocessing described in Section 3.2 remain important confounds.

C.2 Compute and Runtime

The SFT-only control experiment was run on a single NVIDIA H100 GPU with a 12 GB MIG slice for approximately 40 hours. The remaining experiments, including the other SFT and screening runs, were performed on a single Google Colab RTX Pro GPU for approximately 23 hours. No distributed data parallelism or model sharding was used in these experiments.

Table 5: Compute resources used for the experiments. Runtime values are approximate wall-clock runtimes for the corresponding experimental runs.

Experiment	Hardware	Runtime
SFT-only control	1× H100, 12 GB MIG slice	∼40 hours
Other experiments	1× Nvidia RTX Pro 6000 GPU	∼23 hours

All experiments use a single GPU. The GPU model for the SFT-only control was an H100 12 GB MIG slice, while the other experiments were run on a G4 GPU. Runtime differences therefore reflect both the computational workload and the available hardware.

D Assets and Licenses

Assets and licenses. Table 6 lists every external asset we use. All are publicly available, and our use of each falls within its stated terms.

Table 6: External assets used in this work.

Asset	Role	License
SDAR-4B-Chat [Cheng et al., 2025]	Primary model under study	Apache 2.0
TraDo-4B [Wang et al., 2025]	Replication model (Section 3.3)	MIT
DeepMath-103K [He et al., 2025]	Training and evaluation data	MIT

All three licenses permit research use, fine-tuning, and reporting of derived results. We redistribute none of these assets; we release only our evaluation code and the fixed 40-problem benchmark subset (problem indices into the public DeepMath-103K dataset, not the underlying text itself).

NeurIPS Paper Checklist

1. **Claims**

   Question: Do the main claims made in the abstract and introduction accurately reflect the paper's contributions and scope?

   Answer: [Yes]

   Justification: The abstract and introduction state the main findings of the paper and limit the claims to the experiments performed. In particular, we do not claim a final benefit from self-distillation because distillation training is not completed in this work.

   Guidelines:

   - The answer [N/A] means that the abstract and introduction do not include the claims made in the paper.
   - The abstract and/or introduction should clearly state the claims made, including the contributions made in the paper and important assumptions and limitations. A [No] or [N/A] answer to this question will not be perceived well by the reviewers.
   - The claims made should match theoretical and experimental results, and reflect how much the results can be expected to generalize to other settings.
   - It is fine to include aspirational goals as motivation as long as it is clear that these goals are not attained by the paper.

2. **Limitations**

   Question: Does the paper discuss the limitations of the work performed by the authors?

   Answer: [Yes]

   Justification: Section 4 discusses the single 40-problem benchmark, single random seed, use of LoRA on one 4B model, differences between the SFT arms, target truncation, compute limitations, and the fact that final self-distillation performance is not yet measured.

   Guidelines:

   - The answer [N/A] means that the paper has no limitation while the answer [No] means that the paper has limitations, but those are not discussed in the paper.
   - The authors are encouraged to create a separate "Limitations" section in their paper.
   - The paper should point out any strong assumptions and how robust the results are to violations of these assumptions (e.g., independence assumptions, noiseless settings, model well-specification, asymptotic approximations only holding locally). The authors should reflect on how these assumptions might be violated in practice and what the implications would be.
   - The authors should reflect on the scope of the claims made, e.g., if the approach was only tested on a few datasets or with a few runs. In general, empirical results often depend on implicit assumptions, which should be articulated.

- The authors should reflect on the factors that influence the performance of the approach. For example, a facial recognition algorithm may perform poorly when image resolution is low or images are taken in low lighting. Or a speech-to-text system might not be used reliably to provide closed captions for online lectures because it fails to handle technical jargon.
- The authors should discuss the computational efficiency of the proposed algorithms and how they scale with dataset size.
- If applicable, the authors should discuss possible limitations of their approach to address problems of privacy and fairness.
- While the authors might fear that complete honesty about limitations might be used by reviewers as grounds for rejection, a worse outcome might be that reviewers discover limitations that aren't acknowledged in the paper. The authors should use their best judgment and recognize that individual actions in favor of transparency play an important role in developing norms that preserve the integrity of the community. Reviewers will be specifically instructed to not penalize honesty concerning limitations.

3. **Theory assumptions and proofs**

   Question: For each theoretical result, does the paper provide the full set of assumptions and a complete (and correct) proof?

   Answer: [N/A]

   Justification: The paper does not present new theoretical results, theorems, or formal proofs. Its conclusions are based on empirical experiments and analysis.

   Guidelines:

   - The answer [N/A] means that the paper does not include theoretical results.
   - All the theorems, formulas, and proofs in the paper should be numbered and cross-referenced.
   - All assumptions should be clearly stated or referenced in the statement of any theorems.
   - The proofs can either appear in the main paper or the supplemental material, but if they appear in the supplemental material, the authors are encouraged to provide a short proof sketch to provide intuition.
   - Inversely, any informal proof provided in the core of the paper should be complemented by formal proofs provided in appendix or supplemental material.
   - Theorems and Lemmas that the proof relies upon should be properly referenced.

4. **Experimental result reproducibility**

   Question: Does the paper fully disclose all the information needed to reproduce the main experimental results of the paper to the extent that it affects the main claims and/or conclusions of the paper (regardless of whether the code and data are provided or not)?

   Answer: [Yes]

   Justification: The paper and appendix provide the model, benchmark size, decoding schedules, training procedure, LoRA configuration, optimizer, learning rate, batch mechanics, target length, compute setup, and screening settings needed to reproduce the reported experiments.

   Guidelines:

   - The answer [N/A] means that the paper does not include experiments.
   - If the paper includes experiments, a [No] answer to this question will not be perceived well by the reviewers: Making the paper reproducible is important, regardless of whether the code and data are provided or not.
   - If the contribution is a dataset and/or model, the authors should describe the steps taken to make their results reproducible or verifiable.
   - Depending on the contribution, reproducibility can be accomplished in various ways. For example, if the contribution is a novel architecture, describing the architecture fully might suffice, or if the contribution is a specific model and empirical evaluation, it may be necessary to either make it possible for others to replicate the model with the same dataset, or provide access to the model. In general. releasing code and data is often

one good way to accomplish this, but reproducibility can also be provided via detailed instructions for how to replicate the results, access to a hosted model (e.g., in the case of a large language model), releasing of a model checkpoint, or other means that are appropriate to the research performed.

- While NeurIPS does not require releasing code, the conference does require all submissions to provide some reasonable avenue for reproducibility, which may depend on the nature of the contribution. For example
  (a) If the contribution is primarily a new algorithm, the paper should make it clear how to reproduce that algorithm.
  (b) If the contribution is primarily a new model architecture, the paper should describe the architecture clearly and fully.
  (c) If the contribution is a new model (e.g., a large language model), then there should either be a way to access this model for reproducing the results or a way to reproduce the model (e.g., with an open-source dataset or instructions for how to construct the dataset).
  (d) We recognize that reproducibility may be tricky in some cases, in which case authors are welcome to describe the particular way they provide for reproducibility. In the case of closed-source models, it may be that access to the model is limited in some way (e.g., to registered users), but it should be possible for other researchers to have some path to reproducing or verifying the results.

5. **Open access to data and code**

   Question: Does the paper provide open access to the data and code, with sufficient instructions to faithfully reproduce the main experimental results, as described in supplemental material?

   Answer: [Yes]

   Justification: The paper provides a public GitHub repository linked in the abstract, containing the code and data required to reproduce the main experimental results. The repository includes instructions for running the experiments and reproducing the reported results, together with the experimental details provided in the paper and appendix.

   Guidelines:

   - The answer [N/A] means that paper does not include experiments requiring code.
   - Please see the NeurIPS code and data submission guidelines (`https://neurips.cc/public/guides/CodeSubmissionPolicy`) for more details.
   - While we encourage the release of code and data, we understand that this might not be possible, so [No] is an acceptable answer. Papers cannot be rejected simply for not including code, unless this is central to the contribution (e.g., for a new open-source benchmark).
   - The instructions should contain the exact command and environment needed to run to reproduce the results. See the NeurIPS code and data submission guidelines (`https://neurips.cc/public/guides/CodeSubmissionPolicy`) for more details.
   - The authors should provide instructions on data access and preparation, including how to access the raw data, preprocessed data, intermediate data, and generated data, etc.
   - The authors should provide scripts to reproduce all experimental results for the new proposed method and baselines. If only a subset of experiments are reproducible, they should state which ones are omitted from the script and why.
   - At submission time, to preserve anonymity, the authors should release anonymized versions (if applicable).
   - Providing as much information as possible in supplemental material (appended to the paper) is recommended, but including URLs to data and code is permitted.

6. **Experimental setting/details**

   Question: Does the paper specify all the training and test details (e.g., data splits, hyperparameters, how they were chosen, type of optimizer) necessary to understand the results?

   Answer: [Yes]

Justification: The paper describes the fixed evaluation benchmark, decoding schedules, training curriculum, supervision objectives, LoRA settings, optimizer, learning rate, batch size, gradient accumulation, target length, and compute resources. Additional hyperparameters and training details are provided in Appendix-C (C.1).

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- The experimental setting should be presented in the core of the paper to a level of detail that is necessary to appreciate the results and make sense of them.
- The full details can be provided either with the code, in appendix, or as supplemental material.

7. **Experiment statistical significance**

Question: Does the paper report error bars suitably and correctly defined or other appropriate information about the statistical significance of the experiments?

Answer: [No]

Justification: We do not report conventional error bars or statistical significance tests because the main evaluation uses a single 40-problem benchmark and one random seed. We instead explicitly report the approximate binomial uncertainty of ± 15 percentage points and treat the results as evidence of large effects rather than precise estimates.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- The authors should answer [Yes] if the results are accompanied by error bars, confidence intervals, or statistical significance tests, at least for the experiments that support the main claims of the paper.
- The factors of variability that the error bars are capturing should be clearly stated (for example, train/test split, initialization, random drawing of some parameter, or overall run with given experimental conditions).
- The method for calculating the error bars should be explained (closed form formula, call to a library function, bootstrap, etc.)
- The assumptions made should be given (e.g., Normally distributed errors).
- It should be clear whether the error bar is the standard deviation or the standard error of the mean.
- It is OK to report 1-sigma error bars, but one should state it. The authors should preferably report a 2-sigma error bar than state that they have a 96% CI, if the hypothesis of Normality of errors is not verified.
- For asymmetric distributions, the authors should be careful not to show in tables or figures symmetric error bars that would yield results that are out of range (e.g., negative error rates).
- If error bars are reported in tables or plots, the authors should explain in the text how they were calculated and reference the corresponding figures or tables in the text.

8. **Experiments compute resources**

Question: For each experiment, does the paper provide sufficient information on the computer resources (type of compute workers, memory, time of execution) needed to reproduce the experiments?

Answer: [Yes]

Justification: Appendix-C (C.2) reports the GPU type, available GPU memory, number of GPUs, and approximate runtime for the main experimental runs. The SFT-only control used one H100 with a 12,GB MIG slice for approximately 40 hours, while the other experiments used one G4 GPU for approximately 23 hours.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- The paper should indicate the type of compute workers CPU or GPU, internal cluster, or cloud provider, including relevant memory and storage.

- The paper should provide the amount of compute required for each of the individual experimental runs as well as estimate the total compute.
- The paper should disclose whether the full research project required more compute than the experiments reported in the paper (e.g., preliminary or failed experiments that didn't make it into the paper).

9. **Code of ethics**

   Question: Does the research conducted in the paper conform, in every respect, with the NeurIPS Code of Ethics `https://neurips.cc/public/EthicsGuidelines`?

   Answer: [Yes]

   Justification: The work is computational research on language models and mathematical reasoning datasets and does not involve human subjects, personal data collection, or experiments that introduce identifiable risks to participants. We have followed standard research and attribution practices.

   Guidelines:

   - The answer [N/A] means that the authors have not reviewed the NeurIPS Code of Ethics.
   - If the authors answer [No], they should explain the special circumstances that require a deviation from the Code of Ethics.
   - The authors should make sure to preserve anonymity (e.g., if there is a special consideration due to laws or regulations in their jurisdiction).

10. **Broader impacts**

    Question: Does the paper discuss both potential positive societal impacts and negative societal impacts of the work performed?

    Answer: [Yes]

    Justification: The work is foundational research on the reliability and efficiency of diffusion language models. Positive impacts include better understanding of failure modes in efficient language generation, while the main potential negative impact is that improved generative models could be misused; our work does not introduce a new deployment or application with a direct high-risk use.

    Guidelines:

    - The answer [N/A] means that there is no societal impact of the work performed.
    - If the authors answer [N/A] or [No], they should explain why their work has no societal impact or why the paper does not address societal impact.
    - Examples of negative societal impacts include potential malicious or unintended uses (e.g., disinformation, generating fake profiles, surveillance), fairness considerations (e.g., deployment of technologies that could make decisions that unfairly impact specific groups), privacy considerations, and security considerations.
    - The conference expects that many papers will be foundational research and not tied to particular applications, let alone deployments. However, if there is a direct path to any negative applications, the authors should point it out. For example, it is legitimate to point out that an improvement in the quality of generative models could be used to generate Deepfakes for disinformation. On the other hand, it is not needed to point out that a generic algorithm for optimizing neural networks could enable people to train models that generate Deepfakes faster.
    - The authors should consider possible harms that could arise when the technology is being used as intended and functioning correctly, harms that could arise when the technology is being used as intended but gives incorrect results, and harms following from (intentional or unintentional) misuse of the technology.
    - If there are negative societal impacts, the authors could also discuss possible mitigation strategies (e.g., gated release of models, providing defenses in addition to attacks, mechanisms for monitoring misuse, mechanisms to monitor how a system learns from feedback over time, improving the efficiency and accessibility of ML).

11. **Safeguards**

Question: Does the paper describe safeguards that have been put in place for responsible release of data or models that have a high risk for misuse (e.g., pre-trained language models, image generators, or scraped datasets)?

Answer: [N/A]

Justification: The paper does not introduce or release a new high-risk model, image generator, or scraped dataset. It studies existing language models and mathematical data for research purposes and does not require additional release safeguards.

Guidelines:

- The answer [N/A] means that the paper poses no such risks.
- Released models that have a high risk for misuse or dual-use should be released with necessary safeguards to allow for controlled use of the model, for example by requiring that users adhere to usage guidelines or restrictions to access the model or implementing safety filters.
- Datasets that have been scraped from the Internet could pose safety risks. The authors should describe how they avoided releasing unsafe images.
- We recognize that providing effective safeguards is challenging, and many papers do not require this, but we encourage authors to take this into account and make a best faith effort.

12. **Licenses for existing assets**

Question: Are the creators or original owners of assets (e.g., code, data, models), used in the paper, properly credited and are the license and terms of use explicitly mentioned and properly respected?

Answer: [Yes]

Justification: We use three existing assets, each cited at first use in the paper: SDAR-4B-Chat [Cheng et al., 2025] (Apache 2.0), TraDo-4B [Wang et al., 2025] (MIT), and DeepMath-103K [He et al., 2025] (MIT). All three licenses permit research use, fine-tuning, and reporting of derived results, and we redistribute none of the original assets. A full table of assets, roles, and licenses is provided in Appendix D.

Guidelines:

- The answer [N/A] means that the paper does not use existing assets.
- The authors should cite the original paper that produced the code package or dataset.
- The authors should state which version of the asset is used and, if possible, include a URL.
- The name of the license (e.g., CC-BY 4.0) should be included for each asset.
- For scraped data from a particular source (e.g., website), the copyright and terms of service of that source should be provided.
- If assets are released, the license, copyright information, and terms of use in the package should be provided. For popular datasets, `paperswithcode.com/datasets` has curated licenses for some datasets. Their licensing guide can help determine the license of a dataset.
- For existing datasets that are re-packaged, both the original license and the license of the derived asset (if it has changed) should be provided.
- If this information is not available online, the authors are encouraged to reach out to the asset's creators.

13. **New assets**

Question: Are new assets introduced in the paper well documented and is the documentation provided alongside the assets?

Answer: [N/A]

Justification: The paper does not introduce a new dataset, pretrained model, or other standalone research asset. The contribution is an empirical analysis of existing diffusion language models and training procedures.

Guidelines:

- The answer [N/A] means that the paper does not release new assets.
- Researchers should communicate the details of the dataset/code/model as part of their submissions via structured templates. This includes details about training, license, limitations, etc.
- The paper should discuss whether and how consent was obtained from people whose asset is used.
- At submission time, remember to anonymize your assets (if applicable). You can either create an anonymized URL or include an anonymized zip file.

14. **Crowdsourcing and research with human subjects**

    Question: For crowdsourcing experiments and research with human subjects, does the paper include the full text of instructions given to participants and screenshots, if applicable, as well as details about compensation (if any)?

    Answer: [N/A]

    Justification: The paper does not involve crowdsourcing, human participants, participant instructions, or human-subject experiments.

    Guidelines:

    - The answer [N/A] means that the paper does not involve crowdsourcing nor research with human subjects.
    - Including this information in the supplemental material is fine, but if the main contribution of the paper involves human subjects, then as much detail as possible should be included in the main paper.
    - According to the NeurIPS Code of Ethics, workers involved in data collection, curation, or other labor should be paid at least the minimum wage in the country of the data collector.

15. **Institutional review board (IRB) approvals or equivalent for research with human subjects**

    Question: Does the paper describe potential risks incurred by study participants, whether such risks were disclosed to the subjects, and whether Institutional Review Board (IRB) approvals (or an equivalent approval/review based on the requirements of your country or institution) were obtained?

    Answer: [N/A]

    Justification: No human subjects or participants are involved in this research, so IRB approval or equivalent human-subject review is not applicable.

    Guidelines:

    - The answer [N/A] means that the paper does not involve crowdsourcing nor research with human subjects.
    - Depending on the country in which research is conducted, IRB approval (or equivalent) may be required for any human subjects research. If you obtained IRB approval, you should clearly state this in the paper.
    - We recognize that the procedures for this may vary significantly between institutions and locations, and we expect authors to adhere to the NeurIPS Code of Ethics and the guidelines for their institution.
    - For initial submissions, do not include any information that would break anonymity (if applicable), such as the institution conducting the review.

16. **Declaration of LLM usage**

    Question: Does the paper describe the usage of LLMs if it is an important, original, or non-standard component of the core methods in this research? Note that if the LLM is used only for writing, editing, or formatting purposes and does *not* impact the core methodology, scientific rigor, or originality of the research, declaration is not required.

    Answer: [N/A]

Justification: The language models studied in this paper are the objects of the experiments, not tools used to develop the methodology or perform the research analysis. Any LLM assistance used for writing or editing does not affect the core methodology, experimental design, or scientific conclusions.

Guidelines:

- The answer [N/A] means that the core method development in this research does not involve LLMs as any important, original, or non-standard components.
- Please refer to our LLM policy in the NeurIPS handbook for what should or should not be described.